Scuola universitaria professionale
della Svizzera italiana

SUPSI

Course: Numerical computing
Lecturer: Dr. Edoardo Vecchi
Semester: Spring 2026

Student: Ismael Trentin **Discussed with:** Xavier Horisberger

Solution for Project 1 **Due date:** 12 March 2026, 11:59 PM

---

1. Theoretical questions [1.5 points]

(a) **What are an eigenvector, an eigenvalue and an eigenbasis?**

Eigenvectors are non-null vectors that satisfy the following equation:

$$\hat{A}\vec{x} = \lambda\vec{x}$$

where $\hat{A}$ is a square matrix and λ is a scalar number, meaning that the transformation of $\vec{x}$ performed by $\hat{A}$, produces a multiple of $\vec{x}$ by exactly λ times. Knowing this definition, we can easily define that an **eigenvalue** is simply λ, that is, the real scaling coefficient for $\vec{x}$. Since $\lambda \in \mathbb{R}$, there exist an infinite number of eigenvectors for a bounded eigenvalue. To better group a family of eigenvectors, an **eigenbasis** is defined. An eigenbasis for λ is defined as:

$$E_\lambda = \{\vec{v} \mid \hat{A}\vec{v} = \lambda\vec{v}\}$$

(b) **What assumptions should be made to guarantee convergence of the power method?**

Having a matrix $\hat{A}$ of dimensions $n \times n$, $\hat{A}$ must have n linearly independent eigenvectors so that the initial state vector (i.e. guess vector) can be expressed as a linear combination of those eigenvectors. This enables us to factor out the dominant eigenvalue and study the convergence since the other eigenvalues will eventually round off to zero. Therefore, there must be only a single dominant eigenvalue and the initial state vector must also have a non-zero component in the direction of the dominant eigenvalue.

(c) **What is the shift and invert approach?**

The shift and invert approach is a technique that utilizes the inverse iteration algorithm to significantly increase the speed of convergence for eigenvalue computations. The shift consists in subtracting a shift factor α from the main diagonal of $\hat{A}$ so that the new eigenvalues become $(\lambda_j - \alpha)$, while the invert part inverts the matrix resulted from the latter operation $(\hat{A} - \alpha\hat{I})^{-1}$, thus giving the following eigenvalues:

$$\mu_j = \frac{1}{\lambda_j - \alpha}$$

By choosing a factor α that is very close to a specific target λ_t, the value $(\lambda_t - \alpha)$ becomes extremely small. This causes μ_j to become significantly more dominant than the other eigenvalues, reducing the number of iterations needed to perform the computation.

(d) **What is the difference in cost of a single iteration of the power method, compared to the inverse iteration?**

The main difference in the cost comes from the power method using a matrix product as its primary operation, resulting in $O(n^2)$ time complexity, while the inverse iteration needs to

solve linear systems implying $O(n^3)$ for factorization and $O(n^2)$ for substitution, resulting in $O(n^3)$ time complexity. Since the inverse iteration method converges faster, we will still perform computationally slower steps, but with a lower number of iterations.

(e) **What is a Rayleigh quotient and how can it be used for eigenvalue computations?**

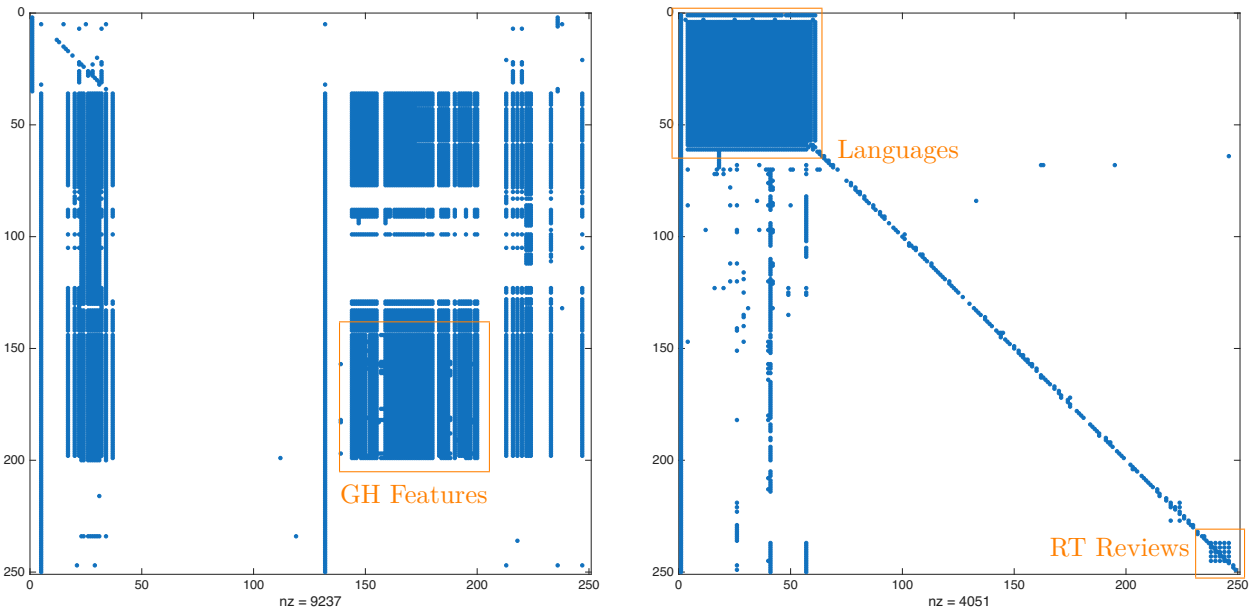
The Rayleigh quotient is a scalar value that provides the best possible estimate for an eigenvalue given a corresponding vector. For a real matrix $\hat{A}$ and a vector $\vec{v}$, it is defined as:

$$\mu(\vec{v}) = \frac{\vec{v}^T \hat{A} \vec{v}}{\vec{v}^T \vec{v}}$$

In eigenvalue computations, it is used as an estimate tool, while in the Rayleigh Quotient Iteration, its value takes place as the shift α factor, making it dynamic.

2. Other webgraphs [0.5 points]

For the first webgraph, the neovim homepage was chosen, while for the second one, the Wikipedia page for 'Steven Universe' was picked. The root URLs passed to `surfer.m` are `https://neovim.io` and `https://en.wikipedia.org/wiki/Steven_Universe`. The script produced the following connectivity matrices:



(a) Neovim Homepage Page Ranks

(b) Wikipedia - Steven Universe Page Ranks

Figure 1: Connectivity Matrices

Based on the provided connectivity matrices, 3 cliques can be easily noticed.

Neovim Homepage

- **GitHub Feature:** a relatively large clique connects all the github features related pages. By having a major number of references pointing to GH, this was clearly to be expected
- **Authority Nodes:** the solid vertical lines represent authority nodes/pages that receive links from almost every other page in the domain. This structural pattern explains why systemic GitHub pages and repositories dominate the top PageRanks alongside core Neovim community links

Wikipedia - Steven Universe

- **Languages Clique:** the large, highly dense block in the top-left of matrix (b), represents a massive clique. In Wikipedia, articles are typically cross-linked to their corresponding translations in other languages. This creates a near-complete graph where almost every language node links to every other language node
- **Rotten Tomatoes Reviews:** the smaller dense block in the bottom-right, correlates directly to the clustered Rotten Tomatoes URLs for the seasons of the show (`s01` through `s05`). These season review pages form a tightly knit clique, heavily cross-linking to one another to facilitate navigation between different seasons of the show

Running the pagerank algorithm on the 2 webgraphs and picking the 10 most relevant ranks for both root URLs produced the subsequent tables:

Rank	URL
0.030588	https://archiveprogram.github.com
0.016507	https://www.binpress.com/vim-creator-bram-moolenaar-interview
0.016507	https://www.youtube-nocookie.com/embed/c4oyfL5o7DU
0.016507	https://bsky.app/profile/neovim.io
0.016507	https://vi.stackexchange.com
0.0097388	https://github.blog
0.0096807	https://support.github.com
0.0096807	https://github.com/pricing
0.008204	https://desktop.github.com
0.0079236	https://github.com/orgs/community/discussions

Table 1: Page Ranks - Neovim Homepage

Rank	URL
0.024353	https://www.latimes.com/entertainment-arts/tv/story/2020-03-25/steven-universe-rebecca-sugar-lgbtq-legacy
0.014794	https://www.polygon.com/2018/5/3/17315264/steven-universe-ending-spoilers-cartoon-network
0.011288	https://www.theverge.com/2017/2/1/14463582/steven-universe-cartoon-network-fantasy-tolerance-empathy
0.011125	https://www.rottentomatoes.com/tv/steven_universe/s05
0.010811	https://www.rottentomatoes.com/tv/steven_universe/s01
0.010811	https://www.rottentomatoes.com/tv/steven_universe/s02
0.010811	https://www.rottentomatoes.com/tv/steven_universe/s03
0.010811	https://www.rottentomatoes.com/tv/steven_universe/s04
0.010689	https://ew.com/article/2015/06/15/steven-universe-creator-growing-gender-politics-her-brother
0.010457	https://www.looper.com/767706/25-shows-like-adventure-time-you-should-watch-next

Table 2: Page Ranks - Wikipedia 'Steven Universe'

3. Connectivity matrix and subcliques [0.5 points]

The organizations responsible for generating the observed cliques can be easily mapped to the used subdomains in the given URLs. Here is the list of the **10** cliques that were identified, mapped to their respective subdomain used by the department:

Start index	Subdomain	Department
71	**baug**	Civil, Environmental and Geomatic Engineering
113	**mat**	Materials
164	**mavt**	Mechanical and Process Engineering
198	**biol**	Biology
221	**chab**	Chemistry and Applied Biosciences
264	**math**	Mathematics
319	**erdw**	Earth Sciences
358	**usys**	Environmental Systems Science
396	**mtec**	Management, Technology, and Economics
436	**gess**	Humanities, Social and Political Sciences

Table 3: ETHZ Cliques

4. Connectivity matrix and disjoint subgraphs [1 point]

The connectivity matrix is a matrix representation of a graph that encodes the edges between nodes. An entry different from zero in the row i and the column j signifies a connection from node i to node j. Since this representation encodes the direction of the edges, we can easily notice that a non-directed graph will produce a symmetrical matrix. This type of matrices inherently have many zero values, and since we are only interested in the edges, we can utilize sparse matrices to save memory space and represent only non-zero values.

The graph in Figure 5 of the original document is thus represented as follows:

$$\begin{array}{cc} (2,1) & 1 \\ (3,1) & 1 \\ (3,2) & 1 \\ (4,2) & 1 \\ (4,3) & 1 \\ (1,4) & 1 \\ (6,5) & 1 \\ (5,6) & 1 \end{array}$$

Given a hyperlink transition probability $p = 0.85$, the PageRanks for the web sub-set represented by U and G are:

$$\vec{x} = \begin{bmatrix} 0.1981 \\ 0.1092 \\ 0.1556 \\ 0.2037 \\ 0.1667 \\ 0.1667 \end{bmatrix}$$

5. PageRanks by solving a sparse linear system [2.5 points]

1. Power method implementation

In the `pagerank1.m` implementation of the power method, the stop condition is identified with a tolerance. By saving a previous state of $\vec{x}$, the error $\epsilon = |\vec{x}_k - \vec{x}_{k-1}|$ can be computed and compared against an arbitrary tolerance τ, in this case $0.1 \cdot 10^{-6}$, to stop the computation when the specified precision is reached.

2. Inverse iteration method implementation

The inverse iteration method in `pagerank2.m` continues to use a tolerance stop condition, but this time the Rayleigh Quotient $\mu(\vec{x})$ was used instead of the error ϵ, defining the new error $\epsilon_R = |\mu(\vec{x})_k - \mu(\vec{x})_{k-1}|$. Since the test is now on the eigenvalues, and by using the Rayleigh Quotient they converge faster than the eigenvectors, we need to prevent the algorithm to stop early by strengthening our tolerance. For this case, $0.1 \cdot 10^{-12}$ was used.

3. Shift values

After running the inverse iteration implementation with different α values, and checking the solutions, it is revealed that the vectors are not always the same. For $\alpha = 0.95$ and $\alpha = 1$, the algorithm converges to the identical and correct PageRank vector because both shift α values are closest to the dominant eigenvalue of 1. However, for $\alpha = 0.8$, the vector is completely different (~ 0.1797) because the shift is closer to the second eigenvalue (0.85), causing it to converge to the wrong eigenvector. For $\alpha = 0.9$, the method fails to converge at all, likely due to the oscillation between the closely clustered eigenvalues.

4. Power vs. Inverse iteration

Comparing the results obtained by calculating the PageRanks for the graphs `web1.m`, `web2.m`, and `web3.m` with the two implementations, it is noticed that both methods always converged with identical results except for the graph `web2.m`, where the power method ranked a different link with the same weight as third place. The power method always took a substantial larger number of iterations in comparison to the inverse iteration method. This is to be expected given what was discussed in section 1, point d. To check if the results were truly equal, the absolute distance of the two computation's output vectors was calculated, and the result found was always a numerical zero.

The following advantages and disadvantages for the **power** method were identified:

Advantages:

- Simplicity: it is straightforward to implement compared to the inverse iteration method
- Low Cost per Iteration: each step is $O(n^2)$, using basic matrix-vector multiplication. Given the sparsity of web graph matrices, this multiplication is fast and memory-efficient.
- High Scalability: it does not solve complex linear systems, making it the standard choice for massive graphs

Disadvantages:

- More Iterations: requires significantly more iterations to converge compared to the inverse iteration method
- Convergence Rate Depends on p: the convergence speed is highly dependent on the damping factor p. As $p \to 1$, the power method takes increasingly longer to separate the dominant eigenvalue from the rest

While for the **inverse iteration** method:

Advantages:

- Less Iterations: by using a shift factor that is very close to the dominant eigenvalue, the algorithm hones in on the correct eigenvector almost immediately, resulting in extremely fast convergence
- Convergence Rate Depends Less on p: the algorithm is much less susceptible to slowdowns caused by high p values

Disadvantages:

- High Cost per Iteration: see section 1, point d
- Poor Scalability: solving massive, real-world like, linear system is computationally unfeasible. Matrix factorization destroys the sparsity of the matrix leading to massive memory consumption
- Sensitivity to α Choice: if α is chosen perfectly, the matrix becomes singular, this non-invertible

rank	rank value	url
1	0.0281	`https://company.wizards.com/legal/wizards-coasts-privacy-policy#donotsell`
2	0.0274	`https://company.wizards.com/legal/wizards-coasts-privacy-policy`
3	0.0235	`https://transformerstcg.hasbro.com/en-us`

Table 4: web1 - power and inverse iteration results

rank	rank value	url
1	0.0493	`https://www.facebook.com/PlayGwent`
2	0.0267	`https://support.gog.com/hc/en-us/articles/115000498685-Cookie-Policy`
3	0.0235	`https://support.gog.com/hc/de/articles/115000498685-Cookie-Richtlinien`

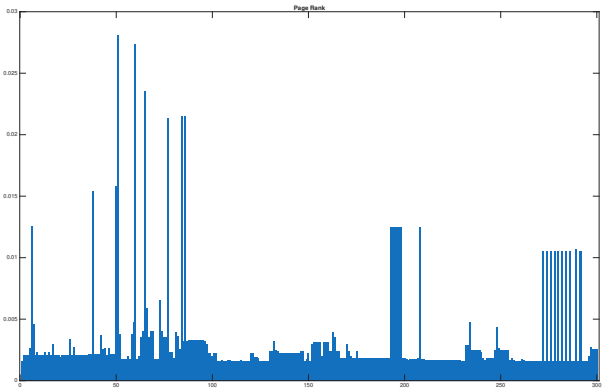
Table 5: web2 - power method results

rank	rank value	url
1	0.0493	`https://www.facebook.com/PlayGwent`
2	0.0267	`https://support.gog.com/hc/en-us/articles/115000498685-Cookie-Policy`
3	**0.0235**	**`https://support.gog.com/hc/fr/articles/115000498685-Politique-des-Cookies`**

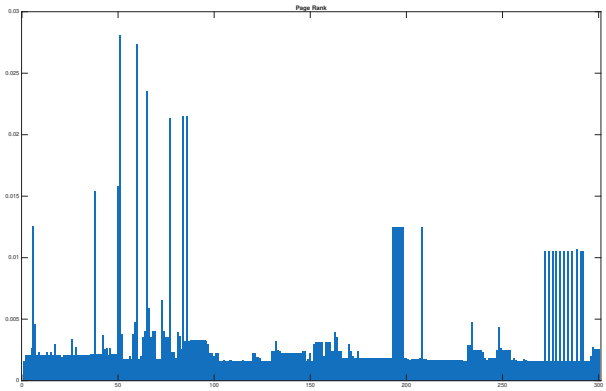
Table 6: web2 - inverse iteration method results

rank	rank value	url
1	0.0350	`https://www.amenitydream.com`
2	0.0276	`https://www.instagram.com/getfandom`
3	0.0262	`https://fandom.zendesk.com`

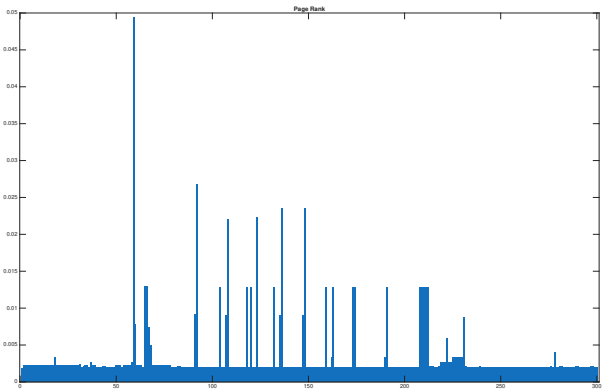
Table 7: web3 - power and inverse iteration results



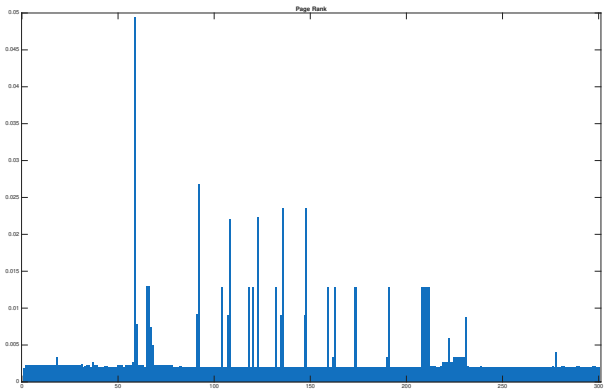
(a) web1 - power method `67` iter.



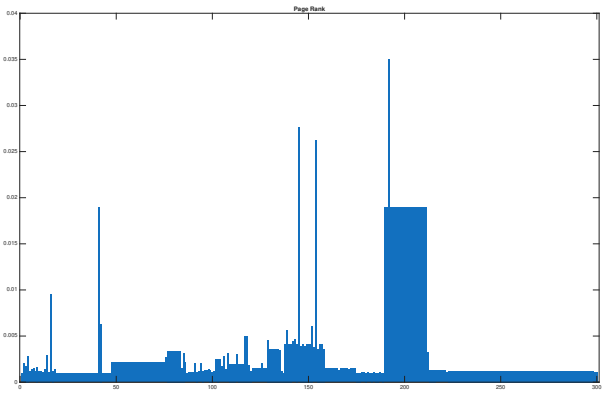
(b) web1 - inverse iteration method `11` iter.



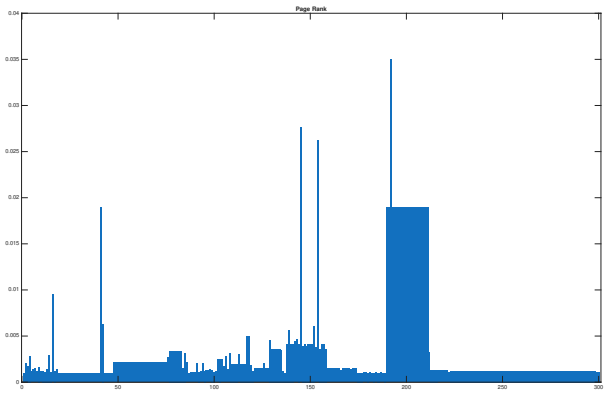
(c) web2 - power method `57` iter.



(d) web2 - inverse iteration method `11` iter.



(e) web3 - power method `68` iter.



(f) web3 - inverse iteration method `10` iter.

Figure 2: `pagerank1.m` and `pagerank2.m` results and iterations

6. Graph perturbations [1.5 points]

1. Degree Centrality

To calculate the 5 nodes with the highest degree centrality, given the graph `WIKI450.m`, the following code was implemented:

```
function [l, R] = degc(G, U, n)
    total_deg = sum(G, 1)' + sum(G, 2);

    [degs_desc, idxs_desc] = sort(total_deg, 'descend');

    top_idxs = idxs_desc(1:n);
    top_degs = degs_desc(1:n);
    top_urls = U(top_idxs);

    l = top_idxs;
    R = table(top_idxs, top_urls, top_degs, ...
        'VariableNames', {'Index', 'URL', 'Degree'});
end
```

which also outputs the following table:

Index	URL	Degree
10	https://species.wikimedia.org/wiki/Main_Page	744
3	https://www.wikidata.org/wiki/Special:EntityPage/Q5296	743
7	https://meta.wikimedia.org/wiki/Main_Page	743
12	https://www.wikidata.org/wiki/Wikidata:Main_Page	742
5	https://foundation.wikimedia.org/wiki/Home	738

Table 8: Wikimedia URL Degrees

2. Nodes Isolation

After removing the incident edges from the 5 nodes obtained from the previous section, their degree centrality is obviously zero. Having changed only 5 nodes out of the 450 total nodes, the change is unnoticible since it corresponds to roughly a 1.1% change in the final matrix.

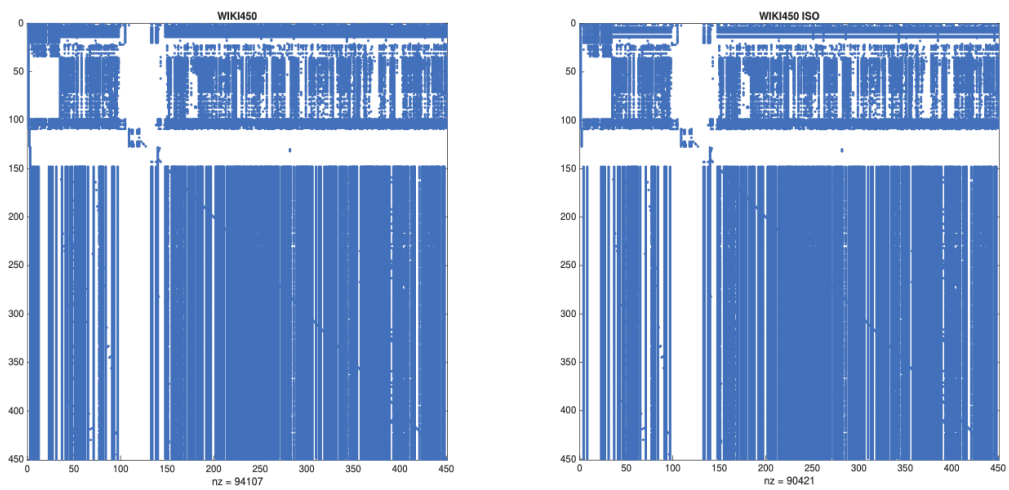


Figure 3: nodes isolation

3. Nodes Impact

To determine the top 10 URLs before and after the nodes isolation, the following function was implemented:

```
function T = pagerank_table(x, U, n)
    [x_desc, idxs_desc] = sort(x, 'descend');

    top_ranks = x_desc(1:n);
    top_idxs = idxs_desc(1:n);
    top_urls = U(top_idxs);

    T = table(top_ranks, top_urls, 'VariableNames', {'Rank', 'URL'})
end
```

Where $\vec{x}$ is the page rank results vector and n the max number of URLs to include in the final ranking. Giving the page rank results, calculated by `pagerank.m`, to the function, the following outputs are produced:

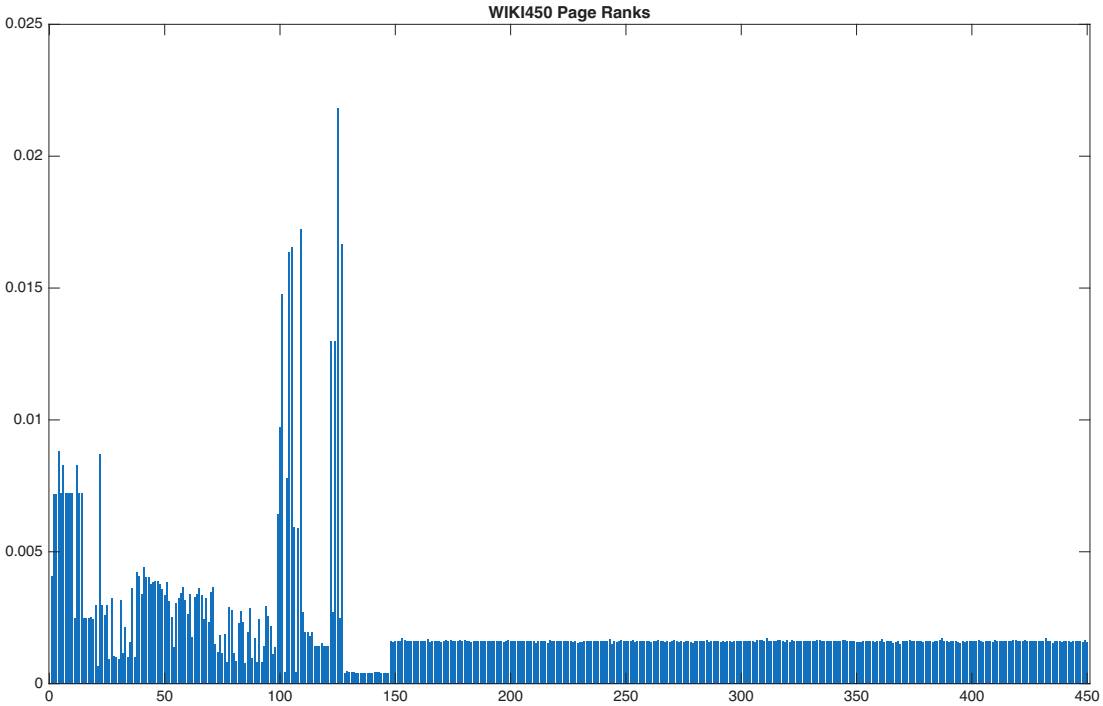
Rank	URL
0.021811	https://www.facebook.com/creativecommons
0.017252	https://creativecommons.org/licenses/by-sa/4.0
0.016674	https://fontawesome.com
0.016540	https://www.mediawiki.org
0.016362	https://wikimediafoundation.org
0.014762	https://developer.wikimedia.org
0.012993	https://mail.creativecommons.org/subscribe
0.012993	https://mastodon.social/@creativecommons
0.0097428	https://foundation.wikimedia.org/wiki/Special:MyLanguage/Policy:Universal_Code_of_Conduct
0.0088171	https://commons.wikimedia.org/wiki/Main_Page

Table 9: top 10 `WIKI450.mat` page ranks

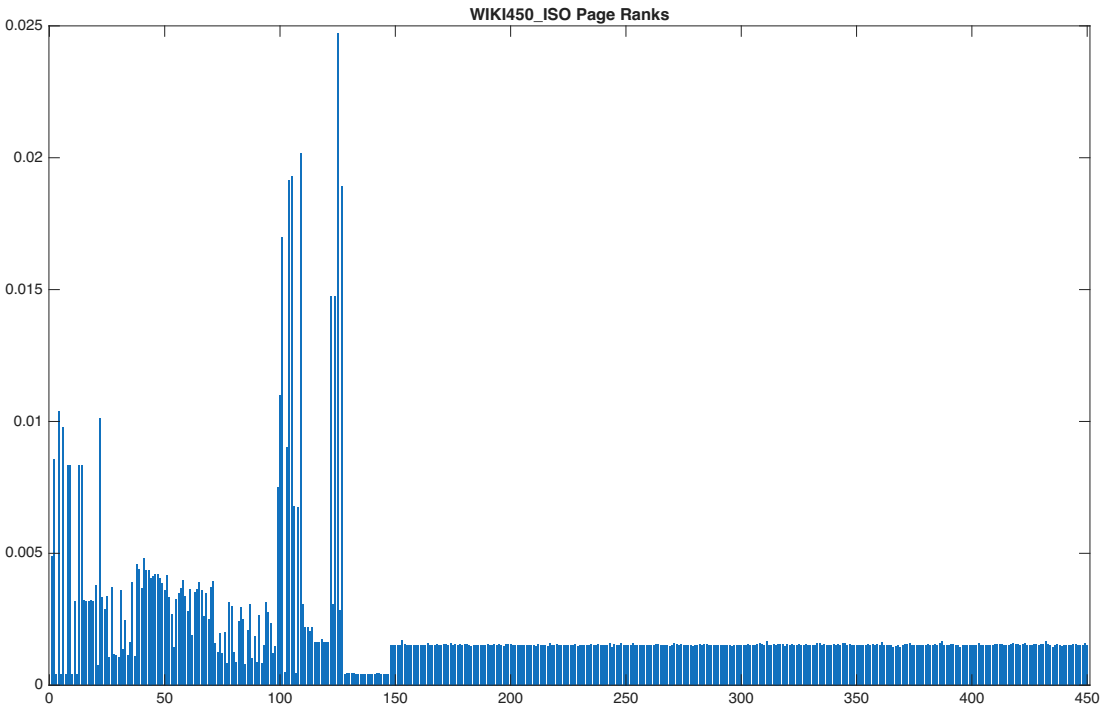
Rank	URL
0.024729	https://www.facebook.com/creativecommons
0.020166	https://creativecommons.org/licenses/by-sa/4.0
0.019312	https://www.mediawiki.org
0.019137	https://wikimediafoundation.org
0.018905	https://fontawesome.com
0.016997	https://developer.wikimedia.org
0.014731	https://mail.creativecommons.org/subscribe
0.014731	https://mastodon.social/@creativecommons
0.010989	https://foundation.wikimedia.org/wiki/Special:MyLanguage/Policy:Universal_Code_of_Conduct
0.010401	https://commons.wikimedia.org/wiki/Main_Page

Table 10: top 10 `WIKI450_ISO.mat` page ranks

When computing the page ranks for `WIKI450.mat` and `WIKI450_ISO.mat`, it can be observed how the isolation of the previous nodes only impacted the ranks that were already influential, leaving



(a) `WIKI450.mat` page ranks



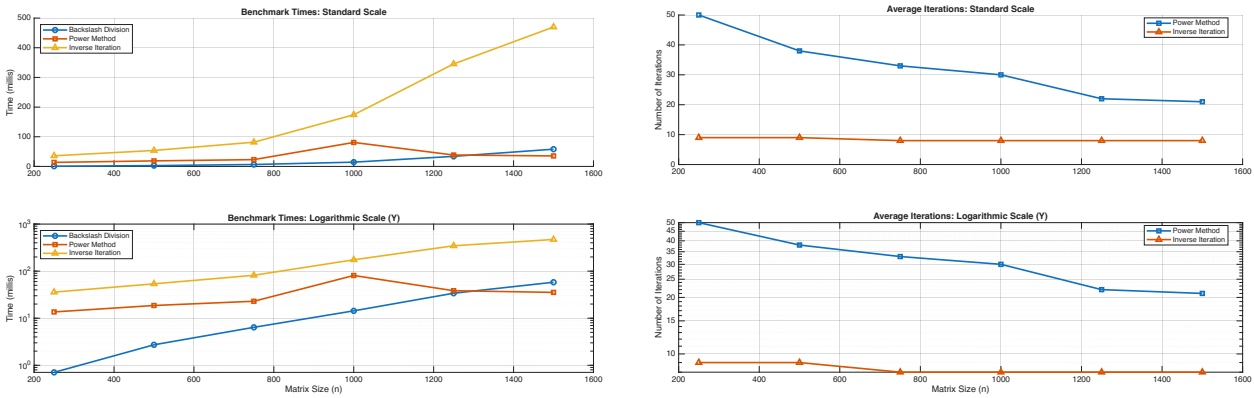
(b) `WIKI450_ISO.mat` page ranks

Figure 4: Before and after node isolation page ranks

the lower ranks untouched. At the beginning of the graph, it is noticeable how a group of nodes had their rank plummet to zero, likely because their influence relied solely on the influence of isolated nodes. It is a ”rich get richer” situation since most of the network’s total influence is concentrated in a restricted number of nodes and influential nodes tend to link to other influential nodes.

7. Introduction to HPC: Algorithmic performance [1 point]

To benchmark the three algorithms implementations, each algorithm implementation was temporarily modified to use the `tic` and `toc` functions returing the the elapsed time and number of iterations, wrapping the core of the algorithm, ignoring the irrelevant steps. To automate this process, the function `benchmark.m` was implemented. The function samples the time performance of the target algorithm 5 times to account for outliers and calculates the average of these samples. The benchmark results for each algorithm and provided matrix, were stored in a 3×6 matrix, one for times and one for iterations, and exported to `bench_times.mat` and `bench_iters.mat`. With these results, Gemini was prompted to generate a matlab function to plot a standard and semi-log scale chart for each algorithm. The results obtained from the benchmarking process, illustrated



(a) Execution Time (b) Iterations

Figure 5: Benchmarks

in Figure 3 and detailed in Table 9, provide clear insights into the computational behavior of the tested algorithms.

Curve Growth and Theoretical Expectations Analyzing the execution time curves, both the Backslash operator and the Inverse Iteration method show a steep, non-linear growth as the matrix dimension n increases. Specifically, the Inverse Iteration time escalates rapidly, reaching 469 ms at $n = 1500$. The Power Method instead, demonstrates a significantly flatter growth trajectory, peaking at only 35 ms. Regarding the number of iterations, the curves remain relatively stable across all matrix dimensions, yet they reveal a strong contrast in magnitude: the Power Method requires an average of 25 to 30 iterations, while Inverse Iteration converges in merely 7 to 9 steps. This behavior aligns perfectly with the expectations, as discussed in section 1, point d. For increasingly large matrices, the $\mathcal{O}(n^3)$ operations dominate the total execution time, explaining the rapid explosion in runtime despite the exceptionally low iteration count.

Scale Comparison and Visualization Advantages The primary advantage of the linear scale is its ability to convey absolute physical quantities. It effectively highlights the real-world computational penalty. The logarithmic scale, in this case semi-logarithmic, becomes highly advantageous when data points span multiple orders of magnitude. By representing relative changes rather than absolute increments, larger values do not take importance away from the smaller ones.

Utility of the Logarithmic Scale This scale is particularly crucial, in this case, for plotting the iterations. Because the Power Method requires roughly one order of magnitude more iterations than Inverse Iteration (~ 50 vs. ~ 8), plotting these on a standard linear scale would compresses the Inverse Iteration data, making it appear as a flat line at zero. The logarithmic visualization proves that both methods maintain a stable iteration count independently of the matrix size n.

	Backslash		**Power Method**		**Inv. Iteration**	
n	**Time**	**Iter.**	**Time**	**Iter.**	**Time**	**Iter.**
250	0.71	—	13.6	50	35.8	9
500	2.72	—	18.6	38	53.8	9
750	6.40	—	22.9	33	81.6	8
1000	14.3	—	80.8	30	174	8
1250	33.9	—	38.4	22	345	8
1500	58.1	—	35.3	21	470	8

Table 11: Benchmarks, Time in ms